

THE SIMPLY-TYPED λ -CALCULUS: FUNCTIONS, EFFECTS AND CALLING MECHANISMS

Alex Kavvos and Samantha Frohlich

Reading: PFPL §8.2

1 Functions

The term $z : \text{Num} \vdash \text{plus}(z; z) : \text{Num}$ expresses the idea of doubling a number. Should we wish to use this term, we must first substitute a number—e.g. `num[57]`—for the free variable z . Instead, we would like our programming language to be able express doubling as a concept itself. That will be achieved by adding **functions**.

2 Functions: Statics

We extend the syntax chart with the following constructs:

types	$\tau ::=$	$\dots$	
		$\tau_1 \rightarrow \tau_2$	function type
pre-terms	$e ::=$	$\dots$	
		$\lambda x : \tau. e$	abstraction
		$e_1(e_2)$	application

The typing is given by the following two rules.

$$\frac{\text{LAM} \quad \Gamma, x : \sigma \vdash e : \tau}{\Gamma \vdash \lambda x : \sigma. e : \sigma \rightarrow \tau}$$

$$\frac{\text{APP} \quad \Gamma \vdash e_1 : \sigma \rightarrow \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash e_1(e_2) : \tau}$$

The first rule creates **λ -abstractions**: it discharges a free variable $x : \sigma$, thereby creating a function which accepts an argument of type σ and returns a result of type τ . Hence, we may express the concept of doubling by

$$\vdash \lambda z : \text{Num}. \text{plus}(z; z) : \text{Num} \rightarrow \text{Num}$$

which is a term of **function type**.

The second rule is known as **application**, and allows the application of a function to a compatible argument.

3 Functions: Dynamics

The dynamics of function types are given by the following rules.

$$\frac{\text{VAL-LAM}}{\lambda x : \tau. e \text{ val}}$$

$$\frac{\text{D-APP-1} \quad e_1 \mapsto e'_1}{e_1(e_2) \mapsto e'_1(e_2)}$$

$$\frac{\text{D-BETA}}{(\lambda x : \tau. e_1)(e_2) \mapsto e_1[e_2/x]}$$

The definition of substitution is the same as before, but extended with the clauses

$$(\lambda y : \tau. u)[e/x] \stackrel{\text{def}}{=} \lambda y : \tau. u[e/x] \quad (e_1(e_2))[e/x] \stackrel{\text{def}}{=} (e_1[e/x])(e_2[e/x])$$

Every λ -abstraction is a value: its body is ‘frozen’ until an argument is provided.

The rule **D-BETA** encapsulates the meaning of functions. If we have a function $\lambda x. e_1$ is applied to an argument e_2 , then we must evaluate the **body** e_1 of the function with the **argument** e_2 substituted for the variable x . This accords with our mathematical experience: if $f(x) \stackrel{\text{def}}{=} x^2$ then $f(5) = (x^2)[5/x] = 5^2$. However, we shall now write the definition using λ -notation, viz. as $f \stackrel{\text{def}}{=} \lambda x. x^2$.

4 Functions: Examples

Our typing rule is the most obvious solution to adding functions. However, it is worth noting that we have perhaps obtained more than we asked: our language now has **higher-order functions**.

For example, we have the following typing derivation.

$$\begin{array}{c}
 \frac{}{x : \text{Num}, y : \text{Num} \vdash x : \text{Num}} \text{VAR} \quad \frac{}{x : \text{Num}, y : \text{Num} \vdash y : \text{Num}} \text{VAR} \\
 \frac{}{x : \text{Num}, y : \text{Num} \vdash \text{plus}(x; y) : \text{Num}} \text{PLUS} \\
 \frac{}{x : \text{Num} \vdash \lambda y : \text{Num}. \text{plus}(x; y) : \text{Num} \rightarrow \text{Num}} \text{LAM} \\
 \frac{}{\vdash \underbrace{\lambda x : \text{Num}. \lambda y : \text{Num}. \text{plus}(x; y)}_{\text{add}} : \text{Num} \rightarrow (\text{Num} \rightarrow \text{Num})} \text{LAM}
 \end{array}$$

This is a function that returns a function. It corresponds to the Haskell definition

```
add :: Integer -> Integer -> Integer
add x y = x + y
```

which can also be written as

```
add :: Integer -> Integer -> Integer
add = \x -> \y -> x + y
```

This definition gives rise to the following transition sequence.

$$\begin{aligned}
 \text{add}(\text{num}[1])(\text{num}[2]) &\mapsto (\lambda y : \text{Num}. \text{plus}(\text{num}[1]; y))(\text{num}[2]) \\
 &\mapsto \text{plus}(\text{num}[1]; \text{num}[2]) \\
 &\mapsto \text{num}[3]
 \end{aligned}$$

The following is also a valid derivation, where $\Gamma \stackrel{\text{def}}{=} f : \text{Num} \rightarrow \text{Num}, x : \text{Num}$.

$$\begin{array}{c}
 \frac{}{\Gamma \vdash f : \text{Num} \rightarrow \text{Num}} \text{VAR} \quad \frac{}{\Gamma \vdash x : \text{Num}} \text{VAR} \\
 \frac{}{\Gamma \vdash f(x) : \text{Num}} \text{APP} \\
 \frac{}{f : \text{Num} \rightarrow \text{Num}, x : \text{Num} \vdash f(f(x)) : \text{Num}} \text{APP} \\
 \frac{}{f : \text{Num} \rightarrow \text{Num} \vdash \lambda x : \text{Num}. f(f(x)) : \text{Num} \rightarrow \text{Num}} \text{LAM} \\
 \frac{}{\vdash \underbrace{\lambda f : \text{Num} \rightarrow \text{Num}. \lambda x : \text{Num}. f(f(x))}_{\text{twice}} : (\text{Num} \rightarrow \text{Num}) \rightarrow (\text{Num} \rightarrow \text{Num})} \text{LAM}
 \end{array}$$

This is a function that both takes in and returns a function. It corresponds to the Haskell definition

```
twice :: (Num -> Num) -> Num -> Num
twice f x = f (f x)
```

This gives rise to the multi-step transition: $\text{twice}(\text{add}(\text{num}[2]))(\text{num}[0]) \mapsto^* \text{num}[4]$.

It is possible to obtain only first-order functions, but it requires additional effort: see PFPL §8.1.

5 STLC

We have completed a presentation of

the constants-and-functions fragment of the simply-typed λ -calculus (STLC)

The constants referred to above amount to the basic language of numbers and strings, which consists of some **base types**—e.g. Num and Str—as well as some **primitive functions**, e.g. `plus(–; –)` and `cat(–; –)`.

The STLC satisfies the usual properties of type safety, namely progress and preservation.

6 Call-by-name vs. Call-by-value

The substitution lemma says that if $\vdash e : \tau$ and $x : \tau \vdash u : \sigma$ then $\vdash u[e/x] : \sigma$. In other words we may replace a free variable $x : \tau$ with any term of that type. Thus, **variables stand for terms**. This evaluation strategy—the only one we have used until now—is known as **call-by-name (CBN)**.

This approach to free variables comes from mathematics. For example, if we define $f(x) \stackrel{\text{def}}{=} x + x$ then we have

$$f(1 + 1) = (1 + 1) + (1 + 1) = 2 + (1 + 1) = 2 + 2 = 4$$

However, there would be a slight advantage in the number of evaluation steps if we simplified the function argument to a value first. In that case we would have the much shorter computation

$$f(1 + 1) = f(2) = 2 + 2 = 4$$

The way to enforce this is to ask that **variables stand for values**, so that a function argument is evaluated before being substituted in the function body. This evaluation strategy is known as **call-by-value (CBV)**.

Of course we know that the order of evaluation does not matter in mathematical or purely functional settings, as either choice leads to the same result. However, it becomes very important when our programs have **effects**.

Call-by-name is strongly associated with purely functional languages with lazy evaluation. A highly optimized form of it known as call-by-need is used in Haskell. Almost all non-pure languages, i.e. languages with effects, use call-by-value. This includes C, Java, Scala, JS, OCaml, Scheme, and so on. (The exception is ALGOL 60.)

7 The call-by-value λ -calculus

The **call-by-value λ -calculus** is a version of the STLC in which all substitutions replace a variable with a value. As a consequence, the changes only affect constructs that use substitution, i.e. sum types and function types.

The changes do not affect the statics of the STLC, as typing remains the same. However, they affect the dynamics, which now behave as follows. **D-INL**, **D-INR**, **D-LET-1** and **D-APP-2** are new; all changes to previous rules are in

red.

$\frac{\text{VAL-INL} \quad v \text{ val}}{\text{inl}(v) \text{ val}}$	$\frac{\text{VAL-INR} \quad v \text{ val}}{\text{inr}(v) \text{ val}}$	$\frac{\text{VAL-LAM}}{\lambda x : \tau. e \text{ val}}$	$\frac{\text{D-LET} \quad v \text{ val}}{\text{let}(v; x. e_2) \mapsto e_2[v/x]}$
$\frac{\text{D-LET-1} \quad e \mapsto e'}{\text{let}(e; x. e_2) \mapsto \text{let}(e'; x. e_2)}$	$\frac{\text{D-INL} \quad e \mapsto e'}{\text{inl}(e) \mapsto \text{inl}(e')}$	$\frac{\text{D-INR} \quad e \mapsto e'}{\text{inr}(e) \mapsto \text{inr}(e')}$	
$\frac{\text{D-CASE-INL} \quad v \text{ val}}{\text{case}(\text{inl}(v); x. e_1; y. e_2) \mapsto e_1[v/x]}$	$\frac{\text{D-CASE-INR} \quad v \text{ val}}{\text{case}(\text{inr}(v); x. e_1; y. e_2) \mapsto e_2[v/y]}$		
$\frac{\text{D-CASE-1} \quad e \mapsto e'}{\text{case}(e; x. e_1; y. e_2) \mapsto \text{case}(e'; x. e_1; y. e_2)}$	$\frac{\text{D-APP-1} \quad e_1 \mapsto e'_1}{e_1(e_2) \mapsto e'_1(e_2)}$		
$\frac{\text{D-APP-2} \quad e_1 \text{ val} \quad e_2 \mapsto e'_2}{e_1(e_2) \mapsto e_1(e'_2)}$	$\frac{\text{D-BETA} \quad v \text{ val}}{(\lambda x : \tau. e)(v) \mapsto e[v/x]}$		

Progress and preservation are also true for the CBV λ -calculus.

8 Examples

The calculational example given above can be reproduced in λ -calculus as follows. In the CBN STLC we have

$$\begin{aligned}
 & (\lambda x : \text{Num. plus}(x; \text{num}[1]))(\text{plus}(\text{num}[1]; \text{num}[1])) \\
 \mapsto_n & \text{plus}(\text{plus}(\text{num}[1]; \text{num}[1]); \text{num}[1]) \\
 \mapsto_n & \text{plus}(\text{num}[2]; \text{num}[1]) \\
 \mapsto_n & \text{num}[3]
 \end{aligned}$$

However, in the CBV STLC we have

$$\begin{aligned}
 & (\lambda x : \text{Num. plus}(x; \text{num}[1]))(\text{plus}(\text{num}[1]; \text{num}[1])) \\
 \mapsto_v & (\lambda x : \text{Num. plus}(x; \text{num}[1]))(\text{num}[2]) \\
 \mapsto_v & \text{plus}(\text{num}[2]; \text{num}[1]) \\
 \mapsto_v & \text{num}[3]
 \end{aligned}$$

The result is the same, but the process is different!

9 Effects

IO, state, failure and non-determinism are all examples of *effects*. Such side-effects widen the chasm between CBN and CBV.

For example, printing is an **observable** difference that distinguishes the two evaluation strategies.

Without side effects CBN and CBV are observationally identical evaluation strategies (but not if we count the number of evaluation steps).